

Jobsheet 8 — Koneksi PostgreSQL

Dokumentasi ini melanjutkan

[dokumentasi jobsheet-07](#)

(PHP Dasar & Form Handling). Jobsheet-08 menutup satu "lubang" penting yang sudah disinggung berkali-kali di dokumentasi sebelumnya: data yang **benar-benar tersimpan**, tidak hilang begitu sesi browser berakhir.

Tentang docs/wireframe.md

File ini **identik persis** dengan

[`docs/wireframe.md` di jobsheet-07](#) —

tidak ada rancangan UI/UX baru di jobsheet ini.

Kenapa Ini Penting?

Ingat catatan yang sudah berulang kali muncul sejak

[dokumentasi jobsheet-07 §3.5](#):

data di `$_SESSION` **hilang** begitu sesi browser berakhir. Jobsheet-08 mengganti sumber data dari `$_SESSION` menjadi **database PostgreSQL** sungguhan — data yang kamu tambahkan sekarang akan **tetap ada** meskipun kamu menutup browser, mematikan komputer, atau kembali lagi besok.

Apa yang Baru di Jobsheet 8?

Sesuai [README.md](#) jobsheet ini:

1. ``sql/01_buku_anggota.sql`` — skema database: perintah SQL untuk membuat tabel buku dan anggota.
2. ``includes/koneksi.php`` — kode PHP yang menghubungkan aplikasi ke database PostgreSQL, memakai **PDO**.
3. ``proses_tambah.php`` (buku & anggota) — `$_SESSION['buku'][] = ...` dari jobsheet-07 diganti `INSERT ... RETURNING id` lewat prepared statement.
4. ``list.php`` (buku & anggota) — sumber data diganti dari `$_SESSION` menjadi `SELECT * FROM ... ORDER BY id DESC`.
5. ``index.php`` — kartu statistik Total Buku/Anggota sekarang `SELECT COUNT(*)` dari database sungguhan, bukan lagi dummy/session.

Daftar Isi

6. [Konsep Dasar Database & SQL](#)
7. [Skema Database: `01\\_buku\\_anggota.sql`](#)
8. [Persiapan Database Sebelum Menjalankan](#)
9. [Koneksi PHP ke Database: `koneksi.php`](#)
10. [Menyimpan Data: Prepared Statement & `INSERT`](#)
11. [Membaca Data: `SELECT`](#)

12. [Rangkuman & Latihan Lanjutan](#)
13. [Lampiran: Instalasi PostgreSQL di Laragon \(Windows\)](#)

Struktur Folder

```

jobsheet-08/
├── index.php                # Kartu statistik dari SELECT
COUNT(*)
├── includes/
│   ├── header.php, footer.php    # Tidak berubah dari jobsheet-07
│   └── koneksi.php              # BARU – koneksi PDO ke PostgreSQL
├── sql/
│   └── 01_buku_anggota.sql        # BARU – skema tabel buku &
anggota
├── buku/
│   ├── list.php                # SELECT * FROM buku, bukan
$_SESSION
│   ├── tambah.php              # Tidak berubah dari jobsheet-07
│   └── proses_tambah.php        # INSERT via prepared statement
├── anggota/
│   ├── list.php
│   ├── tambah.php
│   └── proses_tambah.php        # INSERT via prepared statement
└── docs/wireframe.md           # Identik dengan jobsheet-07

```

Catatan penting dari jobsheet ini:

- Query di sini memakai **prepared statement** (:nama_parameter), bukan menggabung string mentah — ini fondasi keamanan yang akan

diperdalam di Jobsheet 11.

- Kolom `id` sudah ikut ter-fetch lewat `SELECT *`, meski belum dipakai di tampilan — akan dipakai untuk tautan Edit/Hapus mulai Jobsheet 9.
- **Jobsheet ini butuh persiapan tambahan** (menginstal/menjalankan PostgreSQL, membuat database) sebelum bisa dicoba — dibahas lengkap di [bab 3](#).

1. Konsep Dasar Database & SQL

Ini pengenalan pertamamu ke database sungguhan. Kenali dulu istilah-istilah dasarnya sebelum membaca kode PHP yang menghubunginya.

1.1 Kenapa Butuh Database? (Mengingat Kembali Masalahnya)

Perjalanan penyimpanan data SIMPUS-Mini sejauh ini:

Jobsheet	Cara Menyimpan Data	Masalahnya
01-05	Ditulis manual di HTML	Tidak bisa ditambah lewat form sama sekali
06	File data/*.json	Bisa dibaca , tapi tidak bisa ditambah lewat form (ingat dokumentasi jobsheet-06)
07	\$_SESSION	Bisa ditambah dan dibaca, tapi hilang saat sesi browser berakhir (ingat dokumentasi jobsheet-07 §3.5)
08	Database PostgreSQL	Bisa ditambah, dibaca, dan tersimpan permanen

Database menyelesaikan masalah terakhir: data disimpan di program khusus (disebut *Database Management System/DBMS* — PostgreSQL adalah salah satu contohnya) yang berjalan **terus-menerus** di komputer/server, terpisah dari aplikasi PHP-mu sendiri. Kalau kamu menutup browser, bahkan mematikan server PHP-nya sekalipun, PostgreSQL **tetap berjalan** dan datanya tetap tersimpan aman, siap dibaca lagi kapan pun aplikasi PHP-mu terhubung kembali.

1.2 Database Relasional: Tabel, Baris, Kolom

PostgreSQL adalah **database relasional** — data disimpan dalam bentuk **tabel**, mirip konsepnya dengan tabel HTML yang sudah sangat kamu kenal sejak

[dokumentasi jobsheet-01](#):

Istilah Database	Istilah Tabel HTML yang Setara
Tabel (<i>table</i>)	<code><table></code> — misalnya tabel buku
Kolom (<i>column</i>)	<code><th></code> — misalnya kolom judul, pengarang, tahun
Baris (<i>row</i>)	<code><tr></code> — satu baris = satu buku

Bedanya: tabel di database **benar-benar tersimpan** di disk (bukan sekadar tampilan), dan punya **aturan ketat** tentang jenis data apa yang boleh masuk ke tiap kolom (dibahas di [bab 2](#)) — sesuatu yang tidak dimiliki tabel HTML biasa.

1.3 Apa itu SQL?

SQL (*Structured Query Language*) adalah bahasa khusus untuk "berbicara" dengan database relasional — membuat tabel, menyimpan data,

mengambil data, dst. Berbeda dari PHP/JavaScript yang menulis **instruksi langkah demi langkah** (buat variabel, lalu lakukan ini, lalu lakukan itu), SQL lebih bersifat mendeklarasikan apa yang diinginkan, dan database sendiri yang menentukan cara paling efisien mendapatkannya. Empat perintah SQL dasar yang akan kamu temui:

Perintah	Fungsi	Dibahas di
CREATE TABLE	Membuat tabel baru beserta struktur kolomnya	bab 2
INSERT	Menambahkan satu baris data baru	bab 5
SELECT	Mengambil/membaca data	bab 6
UPDATE/DELETE	Mengubah/menghapus data	Belum dipakai di jobsheet ini — menyusul di Jobsheet 9

1.4 Apa itu PDO?

PHP sendiri **tidak otomatis** tahu cara berkomunikasi dengan PostgreSQL — dibutuhkan sebuah "jembatan". **PDO** (*PHP Data Objects*)

adalah lapisan bawaan PHP yang menyediakan **cara seragam** untuk terhubung ke berbagai jenis database (PostgreSQL, MySQL, SQLite, dst.) memakai kode PHP yang mirip, hanya berbeda sedikit di bagian penyambungan awal. Detail cara memakainya di jobsheet ini dibahas di [bab 4](#).

1.5 Kenapa Ada Langkah "Persiapan" Sebelum Menjalankan Jobsheet Ini?

Berbeda dari jobsheet-01 sampai jobsheet-07 yang langsung bisa dicoba (paling banter perlu `php -S localhost:8000` atau Laragon), jobsheet ini butuh **PostgreSQL sungguhan terpasang dan berjalan** di komputermu, plus sebuah database baru yang harus dibuat dan diisi skemanya lebih dulu sebelum aplikasi PHP bisa terhubung ke mana pun. Langkah-langkah persiapan ini dibahas lengkap di [bab 3](#) — jangan lewati bab itu sebelum mencoba menjalankan jobsheet ini sendiri.

2. Skema Database: 01_buku_anggota.sql

File ini adalah **cetak biru** (skema) struktur database — perintah SQL yang menentukan tabel apa saja yang ada dan aturan tiap kolomnya.

2.1 Kode Lengkap

```
-- Jobsheet 8: skema awal database simpus_mini (PostgreSQL)
-- Jalankan setelah membuat database, misal:
-- createdb simpus_mini
-- psql -d simpus_mini -f sql/01_buku_anggota.sql
CREATE TABLE IF NOT EXISTS buku (
    id SERIAL PRIMARY KEY,
    judul VARCHAR(255) NOT NULL,
    pengarang VARCHAR(255) NOT NULL,
    tahun INTEGER NOT NULL,
    isbn VARCHAR(50),
    stok INTEGER NOT NULL DEFAULT 0,
    kategori VARCHAR(50)
);
CREATE TABLE IF NOT EXISTS anggota (
    id SERIAL PRIMARY KEY,
    nama VARCHAR(255) NOT NULL,
    no_anggota VARCHAR(50) NOT NULL UNIQUE,
    alamat VARCHAR(255),
    no_hp VARCHAR(30)
);
```

2.2 Komentar SQL

```
-- Jobsheet 8: skema awal database simpus_mini (PostgreSQL)
```

Baris yang diawali **dua tanda hubung** (--) adalah **komentar SQL** — tidak dieksekusi sama sekali, murni catatan untuk pembaca kode (mirip fungsi `<!-- ... -->` di HTML yang sudah kamu kenal sejak

[dokumentasi jobsheet-06 §2.1](#),

atau // di JavaScript/PHP).

2.3 CREATE TABLE IF NOT EXISTS

```
CREATE TABLE IF NOT EXISTS buku (
    ...
);
```

- ``CREATE TABLE nama_tabel (...)`` — perintah SQL untuk membuat tabel

baru bernama `buku`, dengan struktur kolom yang didefinisikan di dalam tanda kurung.

- ``IF NOT EXISTS`` — pengaman: kalau tabel `buku` **sudah ada** sebelumnya (misalnya kamu menjalankan file ini dua kali tanpa sengaja), perintah ini **tidak akan error** — ia hanya akan melewatinya begitu saja, bukan mencoba membuat ulang tabel yang sudah ada (yang justru akan gagal dan menghapus data yang sudah ada kalau tidak ada pengaman ini).

2.4 Mendefinisikan Kolom: Nama, Tipe, dan Batasan

Setiap baris di dalam tanda kurung `CREATE TABLE` mendefinisikan satu kolom, dengan pola: `nama_kolom TIPE_DATA [batasan tambahan]`.

id SERIAL PRIMARY KEY

- ``SERIAL`` — tipe data khusus PostgreSQL untuk angka yang bertambah otomatis (*auto-increment*): setiap kali baris baru ditambahkan, PostgreSQL sendiri yang mengisi nilai `id`-nya (1, 2, 3, dan seterusnya), kamu **tidak perlu** menentukannya secara manual.
- ``PRIMARY KEY`` — menandai kolom ini sebagai **kunci utama**: nilainya **wajib unik** (tidak boleh ada dua baris dengan `id` yang sama) dan **wajib selalu ada** (tidak boleh kosong). Setiap tabel yang dirancang dengan baik biasanya punya satu kolom primary key — cara paling andal untuk merujuk ke **satu baris spesifik**, akan dipakai untuk fitur Edit/Hapus mulai Jobsheet 9 (ingat catatan di [README.md](#) jobsheet ini).

judul VARCHAR(255) NOT NULL

- ``VARCHAR(255)`` — tipe data untuk **teks**, dengan panjang **maksimal** 255 karakter (`VARCHAR` = *variable character*, artinya panjang sebenarnya boleh lebih pendek dari batas maksimalnya, beda dengan `CHAR` yang selalu memakai panjang tetap). Ingat kolom teks yang sudah kamu kenal sejak [dokumentasi jobsheet-01](#):
judul, pengarang, isbn, kategori semuanya bertipe `VARCHAR` karena isinya teks.

- ``NOT NULL`` — batasan yang mewajibkan kolom ini **selalu terisi**,

tidak boleh kosong/NULL (istilah database untuk "tidak ada nilai sama sekali", beda dari string kosong ""). Perhatikan ini **cocok** dengan atribut `required` pada `<input>` yang sudah kamu kenal sejak [dokumentasi jobsheet-01 §4.4](#) —

kolom `judul`, `pengarang` di sini sama-sama `NOT NULL`, konsisten dengan field `judul/pengarang` yang sejak awal memang wajib diisi.

tahun INTEGER NOT NULL

``INTEGER`` — tipe data untuk **angka bulat** (tanpa desimal). Kolom `tahun` dan `stok` bertipe ini, konsisten dengan tipe `type="number"` yang sudah kamu pakai di HTML sejak [dokumentasi jobsheet-01 §4.4](#), dan `(int)` type casting yang sudah kamu pakai di PHP sejak [dokumentasi jobsheet-07 §4.5](#).

isbn VARCHAR(50) (Tanpa NOT NULL)

Perhatikan kolom `isbn` dan kategori **tidak** diberi `NOT NULL` — artinya kolom ini **boleh kosong** (NULL). Ini konsisten dengan [dokumentasi jobsheet-01 §4.4](#): field ISBN di form memang **tidak** diberi `required` sejak awal (tidak semua buku lama punya ISBN) — aturan database ini **selaras** dengan aturan yang sudah ada di form HTML.

stok INTEGER NOT NULL DEFAULT 0

`DEFAULT 0` — kalau saat INSERT (dibahas di

[bab 5](#)) kolom stok tidak diisi

sama sekali, PostgreSQL otomatis mengisinya dengan 0. Ini jaring pengaman tambahan, meski di praktiknya proses_tambah.php selalu mengirim nilai stok secara eksplisit.

no_anggota VARCHAR(50) NOT NULL UNIQUE

`UNIQUE` — batasan baru yang belum muncul di tabel buku: nilai kolom ini **tidak boleh ada yang sama** di antara semua baris. Ini masuk akal untuk nomor anggota — ingat dari

[dokumentasi jobsheet-01 §6.4](#),

no_anggota (seperti A001) berfungsi sebagai **identitas** anggota, jadi wajib berbeda-beda untuk setiap orang — database sendiri yang akan **menolak** percobaan menyimpan dua anggota dengan no_anggota yang sama persis, tanpa perlu kode PHP tambahan untuk memeriksanya secara manual (meski jobsheet ini **belum** menangani error itu dengan rapi di sisi PHP — ide latihan lanjutan di [bab 7](#)).

2.5 Bagaimana Kolom-Kolom Ini Berhubungan dengan Kode PHP?

Perhatikan **nama setiap kolom** di sini — `judul`, `pengarang`, `tahun`, `isbn`, `stok`, `kategori` untuk tabel `buku`; `nama`, `no_anggota`, `alamat`, `no_hp` untuk tabel `anggota` — **persis sama** dengan nama kunci array asosiatif yang dipakai `proses_tambah.php` sejak [dokumentasi jobsheet-07](#), yang juga sama dengan atribut `name` pada `<input>` sejak [dokumentasi jobsheet-01](#).

Penamaan konsisten ini (sudah disinggung juga di

[dokumentasi jobsheet-06 §3.3](#))

sekarang terbukti manfaatnya di lapisan **paling dalam**: dari HTML, lewat PHP, sampai ke kolom database — satu field yang sama selalu punya nama yang sama di **setiap** lapisan aplikasi.

3. Persiapan Database Sebelum Menjalankan

Bab ini membedah **langkah demi langkah** persiapan yang tertulis di

[README.md](#) jobsheet ini — **wajib** dilakukan sebelum

mencoba menjalankan aplikasi, berbeda dari jobsheet-jobsheet sebelumnya yang bisa langsung dicoba.

3.1 Langkah 1: Pastikan PostgreSQL & Ekstensi PHP Siap

Pastikan PostgreSQL berjalan dan ekstensi PHP `pdo_pgsql` aktif
 (`php -m | grep pgsql`; bila belum ada, aktifkan `extension=pdo_pgsql`
 di `php.ini` lalu restart server).

- **PostgreSQL harus terpasang dan berjalan** di komputermu (ingat dari [bab 1 §1.1](#):

ia adalah program terpisah yang berjalan sendiri, bukan bagian dari PHP). Kalau kamu memakai **Windows dengan Laragon** dan belum pernah memasang PostgreSQL sama sekali, ikuti dulu panduan langkah-per-langkah di [bab 8 \(Lampiran\)](#) sebelum melanjutkan bab ini.

- ``php -m`` menampilkan daftar seluruh **ekstensi** (modul tambahan) yang aktif di instalasi PHP-mu. `pdo_pgsql` adalah ekstensi **spesifik** yang memungkinkan PDO (ingat dari [bab 1 §1.4](#)) berbicara

dengan PostgreSQL — tanpa ekstensi ini aktif, baris

`new PDO("pgsql:...")` di [bab 4](#) akan gagal total.

- ``php.ini`` adalah file konfigurasi utama PHP. Kalau ekstensi `pdo_pgsql` belum aktif, kamu perlu membuka file ini, mencari baris `;extension=pdo_pgsql` (tanda titik koma di depan berarti "dinonaktifkan/dikomentari"), menghapus titik koma itu, menyimpan file, lalu **me-restart** server PHP supaya perubahan konfigurasi terbaca ulang.

3.2 Langkah 2: Membuat Database

```
createdb simpus_mini
```

`**createdb**` adalah perintah command-line bawaan PostgreSQL untuk membuat sebuah **database baru** bernama `simpus_mini` — wadah kosong tempat semua tabel (dan datanya) nanti akan disimpan. Bandingkan dengan istilah `db` di

[`includes/koneksi.php`](#) —

nama `simpus_mini` inilah yang harus **sama persis** dengan yang ditulis di sana.

3.3 Langkah 3: Menjalankan Skema

```
psql -d simpus_mini -f sql/01_buku_anggota.sql
```

`**psql**` adalah program command-line untuk **berinteraksi** dengan PostgreSQL. Perintah ini berarti: "hubungkan ke database `simpus_mini` (`-d simpus_mini`), lalu jalankan seluruh perintah SQL yang ada di file `sql/01_buku_anggota.sql` (`-f ...`).\" Setelah perintah ini berhasil, tabel `buku` dan `anggota` yang sudah dibahas di [bab 2](#) akan benar-benar **ada** di dalam database `simpus_mini`, siap menerima data.

3.4 Langkah 4: Menyesuaikan Kredensial

Sesuaikan kredensial di `includes/koneksi.php` (`$user`, `$pass`) dengan environment lokal.

Kredensial (*credentials*) adalah informasi identitas/otorisasi — di sini, nama pengguna (`$user`) dan kata sandi (`$pass`) yang dipakai untuk masuk ke PostgreSQL di komputermu. Instalasi PostgreSQL yang berbeda-beda punya kredensial default yang berbeda pula (tergantung cara instalasinya) — kalau nilai default `"postgres"/"postgres"` yang sudah ditulis di `koneksi.php` (dibahas di [bab 4](#)) ternyata tidak cocok dengan instalasi PostgreSQL-mu, kamu perlu menggantinya secara manual dengan kredensial yang benar di komputermu sendiri.

3.5 Urutan Ini Penting — Jangan Dibalik

Perhatikan urutan 4 langkah di atas **saling bergantung**: kamu tidak bisa menjalankan skema ([§3.3](#)) sebelum database-nya dibuat ([§3.2](#)), dan aplikasi PHP tidak akan bisa terhubung sama sekali ([bab 4](#)) kalau ekstensi `pdo_pgsql` belum aktif ([§3.1](#)) atau kredensialnya salah ([§3.4](#)).

Kalau salah satu langkah terlewat, gejalanya biasanya muncul sebagai pesan "**Koneksi database gagal: ...**" — dibahas cara membaca pesan error ini di [bab 4 §4.4](#).

4. Koneksi PHP ke Database: koneksi.php

File PHP baru yang menjadi **jembatan** antara kode PHP dan database

PostgreSQL yang sudah disiapkan di [bab 3](#).

4.1 Kode Lengkap

```
<?php
$host = "localhost";
$port = "5432";
$db   = "simpus_mini";
$user = "postgres";
$pass = "postgres";
try {
    $pdo = new PDO("pgsql:host=$host;port=$port;dbname=$db", $user,
    $pass);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    die("Koneksi database gagal: " . $e->getMessage());
}
```

4.2 Lima Variabel Konfigurasi

```
$host = "localhost";
$port = "5432";
```

```
$db = "simpus_mini";
$user = "postgres";
$pass = "postgres";
```

Variabel	Artinya
\$host	Alamat komputer tempat PostgreSQL berjalan. "localhost" berarti "komputer yang sama dengan tempat kode PHP ini dijalankan" — untuk pengembangan lokal, PostgreSQL biasanya berjalan di komputer yang sama.
\$port	"Pintu" jaringan tempat PostgreSQL "mendengarkan" permintaan koneksi. 5432 adalah port standar/default PostgreSQL.
\$db	Nama database yang ingin dihubungi — harus sama persis dengan nama yang dibuat lewat createdb (bab 3 §3.2).
\$user, \$pass	Kredensial login ke PostgreSQL (ingat dari bab 3 §3.4).

4.3 Membuat Koneksi: new PDO(...)

```
$pdo = new PDO("pgsql:host=$host;port=$port;dbname=$db", $user, $pass);
```

- ``new PDO(...)`` — membuat sebuah **objek** PDO baru, yang mewakili satu koneksi aktif ke database. Objek `$pdo` inilah yang nanti dipakai

untuk menjalankan query `SELECT/INSERT` di

[bab 5](#) dan [bab 6](#).

- `"pgsql:host=$host;port=$port;dbname=$db"` — disebut **DSN** (*Data Source Name*), sebuah string yang merangkum **semua** informasi tempat dan jenis database yang ingin dihubungi dalam satu baris teks.

`pgsql`: di depan memberi tahu PDO untuk memakai **driver** PostgreSQL secara spesifik (ingat dari

[bab 1 §1.4](#), PDO bisa

dipakai untuk berbagai jenis database — `pgsql`: inilah yang menentukan jenisnya di sini). Perhatikan variabel PHP (`$host`, `$port`, `$db`) ditulis **langsung di dalam string** yang diapit tanda kutip dua — ini disebut **string interpolation**, PHP otomatis mengganti `$host` dengan nilai sungguhnya (`"localhost"`) saat string ini diproses.

- ``$user``, ``$pass`` — dikirim sebagai **parameter terpisah** (bukan digabung ke dalam string DSN), karena keduanya adalah informasi sensitif yang secara konvensi dipisahkan dari string koneksi.

4.4 Menangani Kegagalan Koneksi

```
try {
    $pdo = new PDO(...);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    die("Koneksi database gagal: " . $e->getMessage());
}
```

- Ingat pola ``try`/`catch`` dari

[dokumentasi jobsheet-06 §1.6](#) —

konsepnya sama persis di PHP: kode yang **berpotensi gagal** (di sini, mencoba membuat koneksi database) dibungkus `try`, dan kalau gagal, `catch` menangkap error-nya supaya program tidak berhenti total tanpa penjelasan.

- ``catch (PDOException $e)`` — `PDOException` adalah **jenis error** spesifik yang dilempar PDO ketika koneksi gagal (misalnya karena PostgreSQL belum berjalan, kredensial salah, atau nama database tidak ditemukan — persis kasus-kasus yang dibahas di [bab 3 §3.5](#)).

- ``die("...")`` — perintah PHP yang **langsung menghentikan** seluruh eksekusi skrip **seketika itu juga**, sambil menampilkan pesan yang diberikan. Ini pilihan yang masuk akal khusus untuk kegagalan koneksi database: kalau database tidak bisa dihubungi sama sekali, hampir semua halaman di aplikasi ini (yang butuh membaca/menulis data) tidak akan bisa berfungsi sama sekali — lebih baik langsung berhenti dengan pesan yang jelas, daripada melanjutkan dan menghasilkan error-error membingungkan di banyak tempat berbeda.

- ``$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);`` — mengatur PDO supaya, **setelah koneksi berhasil**, setiap kegagalan query berikutnya (misalnya `INSERT` yang gagal karena melanggar `UNIQUE` di [bab 2 §2.4](#)) juga akan dilemparkan sebagai `PDOException`, bukan sekadar

diabaikan diam-diam — praktik yang baik supaya masalah database tidak pernah "tersembunyi" tanpa disadari.

4.5 Bagaimana koneksi.php Dipakai di Halaman Lain?

```
require __DIR__ . '/includes/koneksi.php';
```

Perhatikan halaman-halaman yang butuh akses database (`index.php`, `buku/list.php`, `buku/proses_tambah.php`, dst.) memanggil file ini lewat ``require``, bukan ``include`` yang sudah kamu kenal dari [dokumentasi jobsheet-07 §2.2](#).

Bedanya: kalau file yang di-include ternyata **tidak ditemukan**, PHP hanya menampilkan **peringatan** dan tetap **melanjutkan** menjalankan sisa kode (berpotensi menyebabkan error lain yang membingungkan di kemudian hari, karena variabel `$pdo` yang seharusnya ada jadi tidak ada). `require` yang gagal menemukan filenya akan langsung menghentikan skrip dengan **fatal error** seketika itu juga. Karena `$pdo` adalah sesuatu yang **wajib ada** supaya semua query database berikutnya bisa jalan (tanpa `$pdo`, seluruh halaman ini tidak berguna sama sekali), `require` adalah pilihan yang lebih tepat dibanding `include` — mencerminkan bahwa file ini **wajib** berhasil dimuat, tidak seperti `header.php/footer.php` yang meski penting, secara teknis halaman masih "bisa jalan" (meski tidak lengkap) tanpanya.

5. Menyimpan Data: Prepared Statement & INSERT

Ini pengganti langsung dari `$_SESSION['buku'][] = [...]` yang sudah kamu pelajari di

[dokumentasi jobsheet-07 §4.5](#) —

sekarang data benar-benar dikirim ke database, bukan sekadar disimpan di memori sementara.

5.1 Kode yang Berubah di buku/proses_tambah.php

Sebelumnya (jobsheet-07):

```
if (!isset($_SESSION['buku'])) {
    $_SESSION['buku'] = [];
}
$_SESSION['buku'][] = [
    'judul' => $judul,
    'pengarang' => $pengarang,
    'tahun' => (int) $tahun,
    'isbn' => $isbn,
    'stok' => (int) $stok,
    'kategori' => $kategori,
];
```

Sekarang (jobsheet-08):

```

require __DIR__ . '/../includes/koneksi.php';
// ...validasi (tidak berubah dari jobsheet-07)...
$stmt = $pdo->prepare(
    "INSERT INTO buku (judul, pengarang, tahun, isbn, stok, kategori)
    VALUES (:judul, :pengarang, :tahun, :isbn, :stok, :kategori)
    RETURNING id"
);
$stmt->execute([
    'judul' => $judul,
    'pengarang' => $pengarang,
    'tahun' => (int) $tahun,
    'isbn' => $isbn,
    'stok' => (int) $stok,
    'kategori' => $kategori,
]);

```

Perhatikan seluruh **validasi** (*judul* === "", *is_numeric*(tahun), dst.

— ingat dari

[dokumentasi jobsheet-07 §4.3](#))

sama sekali tidak berubah — validasi tetap penting dan berjalan

persis seperti sebelumnya, **sebelum** kode di bawah ini sempat

dijalankan. Yang berubah **hanya** bagian penyimpanan datanya.

5.2 Menyiapkan Query: \$pdo->prepare(...)

```

$stmt = $pdo->prepare(
    "INSERT INTO buku (judul, pengarang, tahun, isbn, stok, kategori)
    VALUES (:judul, :pengarang, :tahun, :isbn, :stok, :kategori)

```

```
RETURNING id"
);
```

Mari bedah pernyataan SQL `INSERT` ini bagian per bagian:

- ``INSERT INTO buku (judul, pengarang, tahun, isbn, stok, kategori)`` —

"tambahkan baris baru ke tabel `buku`, mengisi kolom-kolom ini..."

(ingat struktur tabel `buku` dari

[bab 2](#)).

- ``VALUES (:judul, :pengarang, ...)`` — "...dengan nilai-nilai berikut." Perhatikan yang ditulis di sini **bukan** nilai sungguhnya (seperti `'Laskar Pelangi'`), melainkan **placeholder** bernama, ditandai titik dua di depan (`:judul, :pengarang, dst`).

- ``RETURNING id`` — instruksi khusus PostgreSQL: setelah baris baru berhasil ditambahkan, **kembalikan** nilai kolom `id` yang baru saja dibuat otomatis (ingat `SERIAL` dari [bab 2 §2.4](#)).

Nilai `id` ini **belum dipakai** di jobsheet ini (ingat catatan di

[README.md](#) jobsheet ini), tapi disiapkan sebagai

fondasi untuk fitur Edit/Hapus yang sungguhan mulai Jobsheet 9 — untuk meng-edit/menghapus satu buku spesifik, aplikasi perlu tahu `id`-nya.

- ``$pdo->prepare(...)`` — method PDO yang **menyiapkan** (tapi belum **menjalankan**) query ini. Hasilnya disimpan ke `$stmt` (singkatan dari *statement*), sebuah objek yang mewakili "query yang siap dieksekusi."

5.3 Apa itu Prepared Statement, dan Kenapa Penting?

Prepared statement adalah teknik menjalankan query SQL dalam dua tahap terpisah: **(1) menyiapkan** struktur query dengan placeholder seperti :judul ([§5.2](#)), lalu (2) mengisi placeholder itu dengan **nilai sungguhan** saat dieksekusi ([§5.4](#)) — **terpisah** dari teks query itu sendiri.

Ini **jauh lebih aman** dibanding cara "naif" menggabung nilai langsung ke teks query, misalnya:

```
// JANGAN LAKUKAN INI – hanya contoh kode yang TIDAK aman
$stmt->query("INSERT INTO buku (judul) VALUES ('" . $judul . "')");
```

Kalau \$judul berisi teks yang **sengaja dibuat berbahaya** oleh pengguna jahat (misalnya berisi karakter tanda kutip dan potongan perintah SQL tambahan), cara penggabungan string ini bisa mengubah makna **query SQL itu sendiri** — **kerentanan yang disebut SQL injection**. Dengan prepared statement, nilai yang dikirim lewat `execute()` **tidak pernah** diperlakukan sebagai bagian dari perintah SQL — ia selalu diperlakukan murni sebagai **data**, apa pun isinya, sehingga tidak mungkin mengubah struktur query aslinya. Ingat catatan di [README.md](#) jobsheet ini: ini baru **fondasi**

keamanan, akan diperdalam lebih jauh di Jobsheet 11.

5.4 Menjalankan Query: `$stmt->execute(...)`

```
$stmt->execute([
    'judul' => $judul,
    'pengarang' => $pengarang,
    'tahun' => (int) $tahun,
    'isbn' => $isbn,
    'stok' => (int) $stok,
    'kategori' => $kategori,
]);
```

``$stmt->execute([...])`` benar-benar **menjalankan** query yang sudah disiapkan tadi, mengisi setiap placeholder dengan nilai dari array asosiatif yang diberikan — perhatikan **kunci** array ini ('judul', 'pengarang', dst., **tanpa** titik dua) harus **cocok** dengan nama placeholder di query (:judul, :pengarang, **dengan** titik dua). PDO otomatis mencocokkan keduanya berdasarkan namanya. Setelah baris ini dijalankan, satu baris baru **benar-benar tersimpan** di tabel `buku` — bandingkan dengan versi jobsheet-07 yang "hanya" menyimpan ke array `$_SESSION['buku']` di memori sementara.

5.5 Bagian yang Tidak Berubah Sama Sekali

Sisa kode setelah `execute()` — flash message dan redirect — identik

persis dengan jobsheet-07:

```
$_SESSION['flash'] = ['type' => 'success', 'pesan' => 'Buku berhasil
ditambahkan.'];
header('Location: list.php');
exit;
```

Ini menunjukkan sesuatu yang penting: `$_SESSION['flash']` (pesan sekali-tampil, ingat dari

[dokumentasi jobsheet-07 §5.2](#))

tetap dipakai di jobsheet ini — hanya `$_SESSION['buku']`/

`$_SESSION['anggota']` (data buku/anggota itu sendiri) yang berpindah ke database. `$_SESSION` masih berguna untuk data **sementara** seperti pesan notifikasi, yang memang **seharusnya** hanya bertahan sebentar, berbeda dari data buku/anggota yang seharusnya permanen.

6. Membaca Data: SELECT

Bab terakhir sebelum rangkuman: bagaimana `list.php` dan `index.php` **membaca kembali** data yang sudah tersimpan lewat proses `INSERT` di [bab 5](#).

6.1 buku/list.php: Mengambil Semua Baris

Sebelumnya (jobsheet-07):

```
$daftarBuku = $_SESSION['buku'] ?? [];
```

Sekarang (jobsheet-08):

```
require __DIR__ . '/../includes/koneksi.php';
$daftarBuku = $pdo->query("SELECT * FROM buku ORDER BY id DESC")-
>fetchAll(PDO::FETCH_ASSOC);
```

Mari bedah query SQL-nya:

- **`SELECT \* FROM buku`** — "ambil **semua kolom** (*) dari semua baris di tabel buku" (ingat struktur tabelnya dari [bab 2](#)).
- **`ORDER BY id DESC`** — mengurutkan hasilnya berdasarkan kolom **id**, secara **menurun** (**DESC** = *descending*, dari besar ke kecil). Ingat **id** bertambah otomatis setiap ada baris baru ([bab 2 §2.4](#)) — mengurutkan menurun berdasarkan **id** berarti buku yang paling baru ditambahkan muncul paling atas di tabel. Bandingkan dengan `$_SESSION['buku']` di jobsheet-07 yang selalu menampilkan buku **sesuai urutan ditambahkan** (baru selalu di bawah, karena `[] = ...` menambah ke akhir array) — sekarang urutannya sengaja dibalik supaya data terbaru lebih mudah terlihat.

6.2 Menjalankan Query Sederhana: `$pdo->query(...)`

```
$pdo->query("SELECT * FROM buku ORDER BY id DESC")
```

Perhatikan di sini memakai ``$pdo->query(...)``, bukan

`$pdo->prepare(...)` + `execute(...)` seperti INSERT di

[bab 5](#).

Bedanya: `query()` dipakai untuk perintah SQL yang tidak melibatkan nilai dari luar (tidak ada data pengguna yang perlu disisipkan ke dalam query ini) — query ini selalu persis sama teksnya setiap kali dijalankan, jadi tidak butuh mekanisme placeholder seperti prepared statement. Aturan praktisnya: kapan pun ada nilai dari `$_POST` (atau sumber luar lain) yang perlu masuk ke query, selalu pakai `prepare()` + `execute()` demi keamanan ([bab 5 §5.3](#)) — kalau tidak ada nilai luar sama sekali, `query()` yang lebih sederhana sudah cukup.

6.3 Mengubah Hasil Query Jadi Array PHP: `fetchAll(PDO::FETCH_ASSOC)`

```
->fetchAll(PDO::FETCH_ASSOC)
```

- ``fetchAll(...)`` — mengambil **seluruh** baris hasil query sekaligus, dikembalikan sebagai sebuah array.

- ``PDO::FETCH_ASSOC`` — mode pengambilan yang menentukan **bentuk** tiap baris di dalam array itu: sebagai **array asosiatif**, dengan kunci berupa **nama kolom** ('judul', 'pengarang', dst.) — persis bentuk yang sudah kamu pakai untuk mengakses `$buku['judul']` di [bab 6.4](#) dan yang sudah kamu kenal dari struktur `$_SESSION['buku']` di `jobsheet-07`. Hasil akhirnya, `$daftarBuku` sekarang berbentuk **persis sama** strukturnya dengan `$_SESSION['buku']` sebelumnya — array berisi banyak array asosiatif, satu per buku.

6.4 Menampilkan Hasilnya di Tabel: Tidak Berubah

```
<?php foreach ($daftarBuku as $buku): ?>
<tr>
  <td><?php echo $buku['judul']; ?></td>
  ...

```

Ini bagian yang **paling menenangkan** untuk disadari: kode `foreach` untuk menampilkan tabel ([dokumentasi jobsheet-07 §5.4](#)) **sama sekali tidak perlu diubah**. Karena `fetchAll(PDO::FETCH_ASSOC)` menghasilkan struktur array yang identik dengan `$_SESSION['buku']` sebelumnya, kode yang **menampilkan** data tidak peduli dari mana sebenarnya `$daftarBuku` berasal (session atau database) — inilah manfaat nyata menjaga struktur data tetap **konsisten** di seluruh lapisan aplikasi, seperti yang sudah disinggung di [bab 2 §2.5](#).

6.5 index.php: Menghitung Total dengan COUNT(*)

```
require __DIR__ . '/includes/koneksi.php';
$totalBuku = $pdo->query("SELECT COUNT(*) FROM buku")->fetchColumn();
$totalAnggota = $pdo->query("SELECT COUNT(*) FROM anggota")->fetchColumn();
```

- ``SELECT COUNT(*) FROM buku`` — alih-alih mengambil **seluruh** data buku hanya untuk menghitung jumlahnya (yang akan boros, apalagi kalau datanya sangat banyak), `COUNT(*)` adalah instruksi SQL yang meminta database **sendiri** yang menghitung jumlah barisnya, dan hanya mengembalikan **satu angka** hasilnya.
- ``->fetchColumn()`` — method PDO untuk mengambil satu nilai tunggal dari hasil query (cocok dipakai di sini karena hasil `COUNT(*)` memang cuma satu angka, beda dengan `fetchAll()` yang mengambil banyak baris sekaligus seperti di [§6.3](#)).

Bandingkan dengan jobsheet-07:

```
$totalBuku = count($_SESSION['buku'] ?? []);
```

Fungsi PHP `count()` (menghitung jumlah item array PHP di memori) diganti `SELECT COUNT(*)` (menghitung jumlah baris di database) — dua cara berbeda untuk **tujuan konsep yang sama**: mengetahui berapa banyak data yang ada, hanya sekarang datanya benar-benar berasal dari

database, bukan array sementara di `$_SESSION`.

7. Rangkuman & Latihan Lanjutan

7.1 Rangkuman Keseluruhan Jobsheet 8

Bagian	Konsep yang Dipelajari
Konsep Dasar	Database relasional, tabel/kolom/baris, SQL, PDO
Skema SQL	CREATE TABLE, tipe data (SERIAL, VARCHAR, INTEGER), batasan (PRIMARY KEY, NOT NULL, UNIQUE, DEFAULT)
Persiapan Database	createdb, psql -f, ekstensi pdo_pgsql, kredensial
Koneksi PDO	DSN, new PDO(...), try/catch untuk kegagalan koneksi, require vs include
Prepared Statement & INSERT	prepare(), placeholder :nama, execute(), kenapa lebih aman dari penggabungan string
Membaca Data SELECT	query() vs prepare(), fetchAll(PDO::FETCH_ASSOC), fetchColumn(), COUNT(*)

7.2 Konsep Inti yang Perlu Diingat

14. **Database menyelesaikan masalah "data hilang"** dari jobsheet-07 — data sekarang tersimpan permanen di PostgreSQL, terpisah dari sesi browser mana pun ([bab 1 §1.1](#)).

15. **Skema SQL mendefinisikan aturan data**, bukan cuma nama kolom — NOT NULL, UNIQUE, tipe data yang tepat semuanya membantu menjaga kualitas data langsung dari lapisan database ([bab 2](#)).

16. **PDO adalah jembatan seragam** antara PHP dan berbagai jenis database, dimulai dengan membuat koneksi lewat DSN ([bab 4](#)).

17. **Prepared statement (``prepare()`` + ``execute()``) adalah cara aman** memasukkan data dari pengguna ke query SQL — pakai ini kapan pun ada nilai dari luar (seperti `$_POST`) yang perlu masuk ke query ([bab 5 §5.3](#)).

18. **Struktur array hasil ``fetchAll(PDO::FETCH_ASSOC)`` konsisten** dengan struktur `$_SESSION` sebelumnya — kode yang menampilkan data tidak perlu berubah sama sekali, hanya sumber datanya yang berpindah ([bab 6 §6.4](#)).

7.3 Cara Mencoba Sendiri

19. Selesaikan **semua** langkah persiapan di

[bab 3](#) — ini **wajib** sebelum melangkah lebih jauh.

20. Jalankan `php -S localhost:8000` (atau lewat Laragon), buka `http://localhost:8000/index.php` —

kartu statistik seharusnya menampilkan `0` untuk Total Buku dan Total Anggota (database baru dibuat, belum ada data).

21. Tambahkan satu buku lewat `buku/tambah.php`. Amati kamu diarahkan ke `list.php` dengan buku barumu muncul **di baris paling atas** (ingat `ORDER BY id DESC` dari [bab 6 §6.1](#)).

22. Kembali ke Beranda — amati kartu "Total Buku" sekarang bertambah jadi 1.

23. **Uji persistensi** sesuai catatan di [README.md](#) jobsheet ini: tutup browser **sepenuhnya**, buka lagi, kunjungi `list.php` — buku yang tadi kamu tambahkan **masih ada**. Bandingkan pengalaman ini dengan uji yang sama di

[dokumentasi jobsheet-07 §7.3](#)

langkah ke-6, di mana data **hilang** setelah browser ditutup.

24. Coba tambahkan **dua** anggota dengan `no_anggota` yang sama persis — amati apa yang terjadi (ingat batasan `UNIQUE` dari [bab 2 §2.4](#)).

Kemungkinan besar kamu akan melihat **halaman error PHP mentah**, bukan pesan flash yang rapi — ini kesempatan bagus untuk latihan di [§7.4](#) poin 1.

7.4 Ide Latihan Tambahan (Opsional)

25. Tangani error `UNIQUE` dengan rapi — bungkus `$stmt->execute(...)` di `anggota/proses_tambah.php` dengan `try/catch (PDOException $e)`, lalu set `$_SESSION['flash']` berisi pesan seperti "No. Anggota sudah dipakai, gunakan nomor lain." alih-alih membiarkan error mentah ditampilkan ke pengguna.

26. Tambah kolom baru — misalnya `tanggal\_ditambahkan` `TIMESTAMP DEFAULT NOW()` di tabel `buku` (cari tahu sendiri arti `NOW()`) dan `TIMESTAMP` lewat dokumentasi PostgreSQL), lalu tampilkan kolom itu di `buku/list.php`.

27. Buat query pencarian di server — tambahkan `WHERE judul ILIKE :keyword` (`ILIKE` = pencocokan teks tanpa memandang huruf besar/kecil di PostgreSQL) ke query `SELECT` di `buku/list.php`, dihubungkan dengan kolom pencarian yang sudah ada di HTML — bandingkan dengan filter tabel **sisi klien** yang sudah kamu bangun di [dokumentasi jobsheet-05 §6](#).

28. Migrasi data lama — coba tulis skrip PHP kecil terpisah yang membaca `data/buku.json` dari jobsheet-06 ([dokumentasi jobsheet-06 §3](#)) lalu memasukkan seluruh isinya ke tabel `buku` lewat `INSERT` — latihan bagus untuk memahami bagaimana data lama bisa "dipindahkan" ke database baru.

Kalau ada bagian yang masih membingungkan, coba baca ulang [bab 1](#) sambil mempraktikkan langkah 5 di [§7.3](#) — merasakan sendiri data yang

benar-benar bertahan setelah browser ditutup adalah cara paling meyakinkan untuk memahami kenapa database itu penting.